Introduction

Tree

Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree

AVL tree: height balanced binary search tree

- Searching in a binary search tree may require n comparisons
- The tree could become skewed – degenerate to a list
- AVL tree gives a solution to this problem
- Each node has a balance factor
  - Height(left subtree) – Height(right subtree)
- Difference is maintained within 1 (-1, 0, 1)
- Rotate the tree to keep the balance

© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree: rotations

Case	Configuration	Rotation
LL	Left-Left	Right rotation
RR	Right-Right	Left rotation
LR	Left-Right	Left rotation + Right rotation
RL	Right-Left	Right rotation + Left rotation

Same direction → single rotation
Opposite directions → double rotation

© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree: **LL Rotation**

Example

Insert: 30, 20, 10

The resulting BST is:

```
      30
     /
    20
   /
  10
```

The path from the unbalanced node is:

30 → 20 → 10

Both moves are **left**. Therefore, this is an **LL case**.

Balance factor

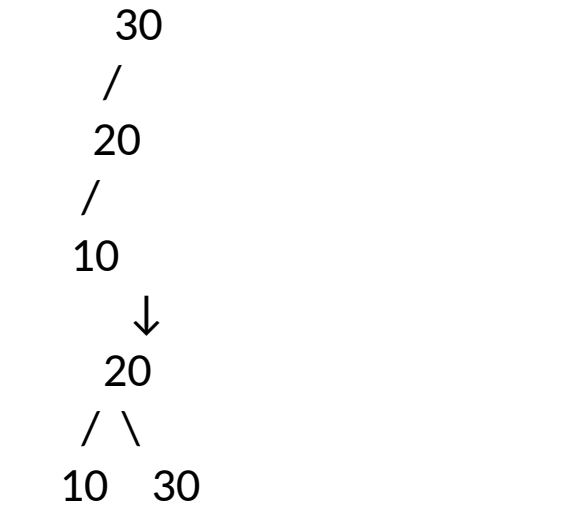
At node 30: Left height = 2; Right height = 0; Therefore: $BF(30) = 2$

The tree is left-heavy

© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree: **LL Rotation**

Solution: Right Rotation : Rotate right around 30:



The tree is now balanced.

LL rule : LL → Right Rotation

© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree: **RR Rotation**

Now consider inserting: 10, 20, 30

The BST becomes:

```
  10
    \
     20
       \
        30
```

The path is:

$10 \rightarrow 20 \rightarrow 30$

Both moves are **right**. Therefore, this is an **RR case**.

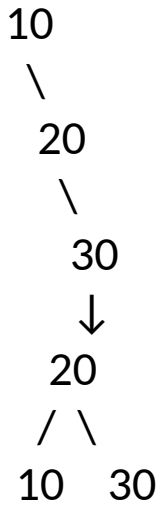
At node 10:

$BF(10) = 0 - 2 = -2$ The node is right-heavy.

© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree: **RR Rotation**

Solution: Solution: Left Rotation; Rotate left around 10:



RR rule

RR → Left Rotation

© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree: **LR Rotation**

Insert: 30, 10, 20

The BST becomes:

```
    30
   /
  10
   \
    20
```

The path is:

30 → 10 → 20

The first move is **left**, but the second is **right**. Therefore:

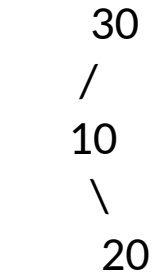
Left → Right

This is an **LR case**. (requires double rotation)

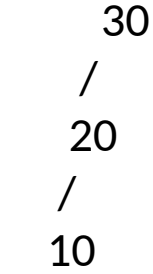
© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree: **LR Rotation**

Step 1 : **Left rotation :** First rotate the subtree rooted at 10 to the left.



becomes:

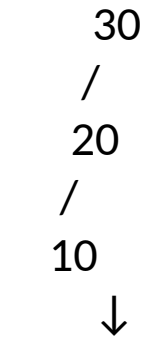


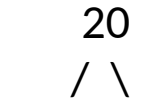
Now the tree has become an LL configuration.

© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree: **LR Rotation**

Step 2: Right rotation : Rotate right around 30:





The tree is balanced.

LR rule : LR → Left Rotation followed by Right Rotation

© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree: **RL Rotation**

Now consider:

10, 30, 20

The BST becomes:

```
   10
    \
     30
    /
   20
```

The path is:

10 → 30 → 20

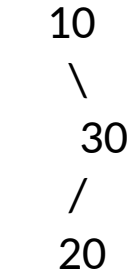
The first move is **right**, followed by **left**.

Therefore, this is an **RL case**. (requires double rotation)

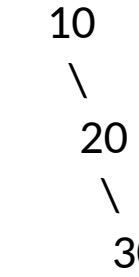
© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree: **RL Rotation**

Step 1: Right rotation : Rotate right around 30:



becomes:

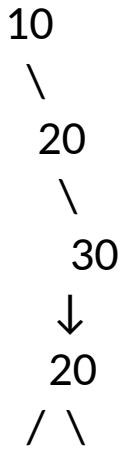


Now we have an RR configuration.

© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree: **RL Rotation**

Step 2: Left rotation : Rotate left around 10:



The tree is balanced.

RL rule : RL → Right Rotation followed by Left Rotation

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)